When Compiler Bugs Meet OS Kernels

HUAZHAO CHEN, Shanghai Jiao Tong University, China
HAO ZHONG*, Shanghai Jiao Tong University, China

A recent study shows that compiler bugs can silently introduce bugs into their compiled code. Although compiler bugs are critical in the development of OS kernels, most studies on compiler bugs analyze the characteristics of only compiler bugs themselves. Their findings are less interesting to outsiders, *e.g.*, the OS community. Although two recent papers have started to analyze how compiler bugs affect the development of other projects, no study has been conducted to explore how such bugs affect OS kernels. As a result, many questions are still open. For instance, in the development of OS kernels, are most compiler bugs identified by OS programmers themselves? When bypassing compiler bugs, what is the relationship between compiler-bug symptoms and modified OS components? The answers to the above questions are useful for both compiler and OS programmers.

To answer the above questions, we conducted the first empirical study on how compiler bugs affect OS kernels. We collected 494 workarounds of compiler bugs from the revision histories of 7 popular OS kernels. We analyze these workarounds and explore the answers to four research questions. We summarized our answers into 9 interesting findings. For instance, we found that in most workarounds, compiler bugs are not reported by OS programmers. As another example, we find that the libraries and Makefiles of OS kernels are the most affected per code line.

ACM Reference Format:
Huazhao Chen and Hao Zhong. 2026. When Compiler Bugs Meet OS Kernels. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (March 2026), 21 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Software infrastructures are the foundation of application development. For instance, as a typical software infrastructure, operating systems (OSes) provide programmers with numerous application programming interfaces (APIs) for managing hardware and software resources, *e.g.*, memory, and devices [25]. According to a recent study [31], a project is likely to trigger compiler bugs if it has more than a half million lines of code. As OSes have millions of lines of code, they can trigger compiler bugs, and such bugs can further introduce errors into compiled OSes. As OS bugs can affect many applications, understanding the affection and effects of compiler bugs in OS kernels is critical for both OS and compiler programmers.

Due to the importance of compiler bugs, researchers [37, 38, 42] conducted various empirical studies to understand compiler bugs. In particular, by analyzing compiler bug reports and their patches, researchers summarize findings about the buggy locations [42], repair patterns [42, 46], and causes [38, 42] for compiler bugs. Although these findings are useful, their audience is narrow,

*Manuscript received July, 2024.
(Corresponding author: Hao Zhong.)

Authors' Contact Information: Huazhao Chen, lyican53@sjtu.edu.cn, Shanghai Jiao Tong University, Shanghai, China; Hao Zhong, zhonghao@sjtu.edu.cn, Shanghai Jiao Tong University, Shanghai, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2026/3-ART
https://doi.org/XXXXXXX.XXXXXXX

and only the developers of compilers and researchers can use such findings in detecting and repairing compiler bugs. Recently, researchers [31, 45] have started to analyze the far-reaching impacts of compiler bugs. These studies are beneficial for both compilers and other projects. For instance, Zhong [45] finds that about half of compiler bugs in real development are unfixed, but He and Zhong [31] report that the most frequently fixed types of compiler bugs are not the most frequent compiler bugs that programmers encounter in real development. Compiler developers can pay more attention to those ignored compiler bugs. As another example, He and Zhong [31] report the patterns of bypassing compiler bugs. Programmers can learn how to bypass compiler bugs. Although their findings are beneficial, they do not analyze how compiler bugs affect the development of OS kernels. OS kernels have millions of lines of code and heavily use the `asm volatile`. In addition, their source files are compiled with unusual flags, *e.g.*, `-nostdlib`. This flag eliminates the buffering layer of standard libraries and the kernel startup code, causing the kernel code to be more directly exposed to the low-level compiler behavior and bugs. As a result, compiler bugs can have a deeper impact on OS kernels, especially when OS bugs can affect many applications and their development. As the prior studies do not focus on OS kernels, many questions in this regard are still open. For instance, are most compiler bugs identified and reported by OS kernel programmers themselves, since they are experts? When compiler bugs are bypassed, what are the relationships between symptoms, OS components, and required modifications?

The answers to the above open questions are useful for both OS and compiler programmers. It could take years to fix critical compiler bugs [45]. As OS kernels are widely reused, OS developers must handle such bugs promptly to reduce their impact. Before compiler bugs are fixed, OS developers can implement workarounds to bypass such bugs. In our ICSE 2026 poster paper [28], we conducted the first empirical study to understand how compiler bugs affect OS kernels and reported our early results. For instance, we find that the knowledge flow is typically from compiler bugs to OS kernels. In other words, most bypassed compiler bugs are not reported by OS developers. In addition, we outlined our plan to extend our poster paper. In this manuscript, we complete our plan and explore the following research questions:

- **RQ1: Who reports the compiler bugs?**
  Motivation. This research question clarifies the knowledge flow directions of bypassing compiler bugs. The answers are useful for understanding the roles of compiler and OS programmers when bypassing compiler bugs.
  Result. Finding 1 shows that most compiler bugs are not reported by OS programmers, and Finding 2 shows that OS programmers care more about wrong-code and optimization compiler bugs.
- **RQ2: How long does it take from reporting bug reports to bypassing them?**
  Motivation. This research question analyzes to what degree compiler bugs affect projects. Before programmers of a project implement the workarounds for a compiler bug, it will affect this project. This research question measures the time interval between reporting compiler bugs and bypassing such bugs in OS kernels under different knowledge flows.
  Result. Finding 5.2.2 shows that it takes 10 times longer to identify and bypass compiler bugs that are not reported by OS programmers than those reported by OS programmers. Finding 4 shows that crashes, optimization, and wrong-code bugs have shorter time intervals.
- **RQ3: How similar are the code samples from bug reports and the real code from OS workarounds?**
  Motivation. This research question explores the similarity between the code samples in a compiler bug report and the modified source files in its corresponding OS kernel workaround. The answers are useful for OS programmers to identify and reduce the triggering code of

compiler bugs in OS kernels.
Result. Finding 5 shows that most code samples in bug reports are dissimilar to OS code, but a few code samples of bug reports are quite similar. Finding 6 shows that the code samples of wrong-code bugs are more different from the triggering code of OS kernels than other symptoms.

- **RQ4: What is the relationship between compiler-bug symptoms, modified OS components, and modified lines of workarounds?**
Motivation. This research question focuses on how different symptoms affect OS kernel components and the modified code lines of corresponding workarounds. The answers are useful for OS and compiler programmers to allocate the resources to handle the impact of compiler bugs or to fix these bugs.
Result. Finding 7 shows compiler bugs tend to affect larger components and the build files. Finding 8 shows that workarounds in the development, the library, and the kernel core have more modifications, and some modifications are repetitive. Finding 9 shows that bypassing crashes and wrong code compiler bugs requires more modifications than other compiler bugs, and modifying the Makefile is unlikely to bypass crash bugs.

2 Comparison with Prior Study

Researchers [31, 45] have conducted empirical studies to understand workarounds of compiler bugs. In this section, we compare our study with prior ones.

2.1 Subject

Although prior studies [31, 45] analyze the workarounds of compiler bugs, they collect workarounds from open-source projects. In contrast, we collect workarounds from the revision histories of OS kernels. As OS kernels is a fundamental infrastructure for many software systems and applications, the impact on OS kernels is more relevant than that on other software systems.

2.2 New RQ and Finding

None of our RQs is explored by the prior studies [31, 45]. As a result, we derive new findings that are not reported by any prior studies. For instance, for the first time, we find that OS programmers care more about wrong-code and optimization compiler bugs (Finding 2). As another example, we are not limited to symptom frequencies but use symptoms to explore more questions. For the first time, we analyze how the symptoms are correlated with OS components (Findings 7 and 8). Besides new findings, we also derive some different findings. For instance, He and Zhong [31] report that most bypassed compiler bugs are diagnostic bugs, but we observe that OS developers bypass more wrong-code bugs (Finding 2). OS developers care about security issues, but wrong code bugs can introduce bugs silently. The complex and low-level code in OS kernels may be more likely to trigger wrong-code bugs. As a result, such bugs may be more likely to be noticed and bypassed in OS development. In addition, our work yields new findings that are not reported by the prior research. For instance, we report how compiler bugs affect the components of OS kernels (Findings 7, 8, and 9).

2.3 Protocol

He *et al.* [31] manually inspect workarounds, and Zhong [45] implement a tool to support their analysis. Our study is mixed. Although we also analyze workarounds of compiler bugs, our research questions are novel and unexplored by the prior studies. For instance, we analyze relationship between compiler bug reporters and programmers of workarounds for the first time. As another example, as OS kernels have similar code structures, we can analyze how compiler bugs affect

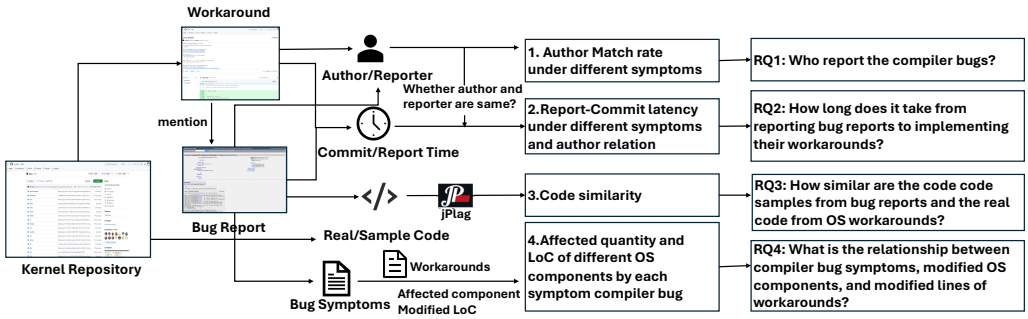


Fig. 1. Overview

different components. As a comparison, the prior studies cannot explore this research question since open source projects have heterogeneous code structures.

2.4 Mutually Corroborating Findings

Despite differences in scope, several of our findings reinforce observations from prior studies [31, 45]. First, as they did, we notice that programmers must bypass compiler bugs since many compiler bugs remain unresolved for many days (Finding 5.2.2). Second, He and Zhong [31] found that many workarounds involve modifying the source code and build configurations. In our study, we find similar patterns (Finding 7). Third, our RQ4 and Zhong [45] calculate modified code lines of compiler-bug workarounds. Zhong groups workarounds by the components of their mentioned compiler bugs, but we group workarounds by the components of their affected OS kernels. In both settings, Zhong [45] and we find that most workarounds modify only 10 to 20 code lines (see Figure 8 for details). As another example, like He and Zhong [31], we group compiler bugs by their symptoms. Our top three symptoms are the same as reported by He and Zhong [31]. Still, the rankings of the symptoms are different, and the difference can reflect the preference of OS programmers (see Figure 8 for details). Our findings are sound since they corroborate with those of prior studies.

In summary, our study is novel, and our findings are sound.

3 Motivating Example

Section 2 compares our study with prior studies from various perspectives such as subjects, research questions, protocols, and findings. In this section, we use concrete examples to further illustrate how our study is different from prior ones. OS programmers report a `llvm` bug [2]. When programmers compile a kernel with `clang-10`, it produces a warning message:

warning: stack frame size of 1368 bytes in function...

Although this is only an unnecessary warning, in the follow-up comment, a compiler programmer confirms that `clang-10` produces broken optimized code due to this bug:

clang-10 appears to have a broken optimization stage that doesn't enable the compiler to prove at compile time that certain memcpys are within bounds, and thus the outline memcpy is always called, resulting in horrific performance, and in some cases, excessive stack frame growth.

Xu *et al.* [43] analyze the security OS bugs that are caused by compiler bugs. Their study does not analyze how to bypass compiler bugs. As introduced in Section 8, most empirical studies analyze the characteristics of compiler bugs. Only two recent studies [31, 45] have started to analyze how compiler bugs affect other projects. Compared with their studies, we focus on OS kernels

and explore new research questions. For instance, we are the first to analyze the knowledge flow between compilers and OSes.

Compiler bug knowledge flows from OSes to compilers when OS programmers identify and report compiler bug to compilers during the OS development. In the example above, Linux programmers identified and reported this compiler bug [2] to `llvm` on May 5, 2020. According to the bug report [2], if `CONFIG_FORTIFY_SOURCE` is enabled, clang will fail to recognize some `memcpy` calls and will not inline them correctly. The compiler bug leads to unnecessarily large stack frames and extra runtime overhead. Due to this bug, the programmers of Linux encounterred an compilation error:

net/bridge/br_multicast.c:2422:6: error: stack frame size of 1472 bytes in function 'br_multicast_get_stats' [-Werror,-Wframe-larger-than=]

Twenty-four days after the bug was reported, Linux programmers implemented a workaround [3]:

```
net/bridge/br_multicast.c
@@ -2413,7 +2413,8
- static void mcast_stats_add_dir(u64 *dst, u64 *src)
+ /* noinline for https://bugs.llvm.org/show_bug.cgi?id=45802#c9 */
+ static noinline_for_stack void mcast_stats_add_dir(u64 *dst, u64 *src)
{
  dst[BR_MCAST_DIR_RX] += src[BR_MCAST_DIR_RX];
  dst[BR_MCAST_DIR_TX] += src[BR_MCAST_DIR_TX];
}
```

Linux programmers bypassed the compilation error with the added `noinline_for_stack` attribute. This attribute asks compilers not to inline the affected method.

Alternatively, compiler bug knowledge flows from compilers to OSes when OS programmers learn from compiler bug reports reported by other programmers and bypass these compiler bugs if they affect OS kernels. For example, an another `llvm` bug #55520 [11] was reported by a non-OS programmer and the description is:

llvm 14 warns with unaligned access for the packed struct.

Linux programmers also observed a similar warning in `include/uapi/linux/videodev2.h`:

./include/uapi/linux/videodev2.h:1810:2: warning: field within 'struct v4l2_ext_control' is less aligned than 'union v4l2_ext_control::(anonymous ...)' ... can lead to unaligned accesses [-Wunaligned-access]

After identifing that this warning is related to this bug [11], Linux programmers implemented a workaround [16] and explicitly linked this warning to this bug [11]:

```
include/uapi/linux/videodev2.h
@@ -1842,7 +1842,7 @@ struct v4l2_ext_control {
- };
+ } __attribute__ ((packed));
 } __attribute__ ((packed));
```

Linux programmers bypassed the allignment warning by adding `packed` to the inner union. Since the outer struct is already `packed`, the memory layout does not change. After applying this workaround, the inner and outer types use the same packing rule. Consequently, compiler will no longer report this alignment warning.

In our study, we analyze which flow direction is more popular. In addition, we calculate the time interval between reporting compiler a bug report and implementing its workaround. We also analyze how the knowledge flow direction affects the time interval. Our study includes other interesting and unexplored analyses. The prior studies [31, 45] do not touch the above research questions. In our study, we collected 494 workarounds from 7 popular OS kernels. He and Zhong [31] and Zhong [45] analyzed 806 and 644 workarounds, respectively. As we focus on a specific type of software, we collected fewer workarounds, but the sample size is comparable to the prior studies.

Table 1. Dataset

Kernel	Workaround	Report	Link	MLOC	Date
Linux	340	231	367	28.29	Sep 4, 2011
freebsd	109	53	115	21.44	Sep 5, 2011
Serenity	13	14	14	1.15	Dec 2, 2018
ReactOS	11	7	13	8.77	Oct 3, 2017
openbsd	10	9	10	22.56	Aug 30, 2016
zephyr	9	4	9	2.67	May 26, 2016
AROS	2	2	2	2.67	May 17, 2019
Total	494	320	530	87.55	–

4 Methodology

This section introduces the dataset (Section 4.1) and the overview of our analysis (Section 4.2).

4.1 Dataset

Table 1 shows our subjects. The column "Kernel" lists the names of our selected OS kernels. The Linux [22] kernel is a widely used open-source operating system kernel known for its robustness, performance, and extensive community support. The FreeBSD [13] kernel is part of a Unix-like operating system, renowned for its advanced networking and security features. The Serenity [20] kernel powers SerenityOS, focusing on simplicity and elegance in its design. The ReactOS [19] kernel aims to be binary-compatible with Microsoft Windows NT, providing a free alternative. The OpenBSD [18] kernel emphasizes security, reliability, and portability, making it suitable for secure environments. The Zephyr [21] kernel is a real-time operating system designed for embedded devices, offering modularity and scalability. AROS [12] is an open-source operating system compatible with AmigaOS, known for its flexibility and modern design. Almost all collected kernels contain workarounds related to `gcc` and `clang`, except for ReactOS, which collected three workaround records related to the `msvc` compiler on this project. The column "Workaround" lists the total number of workarounds whose messages mention compiler bugs. Following the pioneer studies [31, 45], we used the following keywords to query commits from the revision histories of OS kernels to find workarounds of compiler bugs:

gcc.gnu.org/bugzilla/show_bug.cgi?id=

llvm.org/bugs/show_bug.cgi?id=

github.com/llvm/llvm-project/issues

The URLs of compiler bug reports must contain the above keywords. If a commit message contains these URLs, it should be a workarounds that bypass compiler bugs. Our tool retrieved 496 workarounds. After manual inspection, we removed two irrelevant workarounds. In particular, a Linux workaround 2947a45 [15] has a commit message: "update LLVM Bugzilla links". This workaround does not bypass compiler bugs. In addition, a Serenity workaround 73fab93 [7] has a message: "Toolchain: Update to the latest gcc release 10.3.0". This workaround does not bypass compiler bugs either. The column "Report" lists the number of compiler bug reports that are mentioned in the workarounds of OS kernels. As a compiler bug can be mentioned in more than one workaround, we collected more workarounds than bug reports.

When programmers bypass compiler bugs, they may not mention their URLs. As a result, we do not select all the workarounds for compiler bugs. Although we have collected more than 400 subjects, programmers may implement more workarounds. The influence of compiler bugs could be greater than what we analyzed.

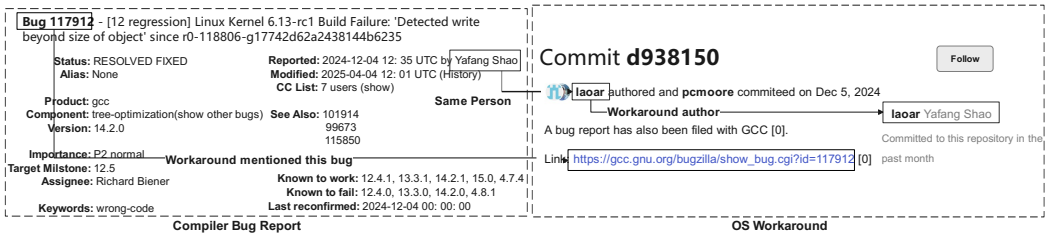


Fig. 2. Compiler bug reporter and OS-kernel workaround programmer

The column "Link" lists the identified links between OS workarounds and compiler bug reports. The links are more than the workarounds since a workaround can mention more than one compiler bug. The column "MLOC" lists lines of code in millions for the corresponding OS kernels. The column "Date" denotes the creation date of their GitHub repository. Please note that the starting dates can be earlier.

4.2 General Protocol

Figure 1 shows the layout of our study. For RQ1, we analyze the knowledge flow directions between OS-kernel developers and compiler developers. We calculate the cases of both directions. For RQ2, we calculate the time interval between the commit time of an OS-kernel workaround and the reporting time of the corresponding compiler bug that the workaround mentions. Meanwhile, we calculate the difference between the two knowledge flow directions. For RQ3, we compare code samples of compiler bug reports with the modified source files of OS-kernel workarounds. We use a tool called JPlag [36] to calculate code similarity. JPlag [36] compares the token sequence of two source files to calculate their similarity. Its tokenizer removes all whitespace and comments and encodes variable names, function names, class names, and literal values to abstract tokens. Although its paper is published in 2002, it is under active maintenance. We select its latest version that was released on February 10, 2025. As a comparison, JPlag is off the shelf and can detect clones in the wild. As a result, we select JPlag in our study. Indeed, all clone detecting tools should be sufficiently accurate if they detect similar clones, *i.e.*, Types 1, 2, and 3 clones. Although a few recent tools can detect Type 4 clones, most tools are designed to run only on benchmarks. We did not select them since it take non-trivial effort to extend them to detect clones in the wild. For RQ4, we group workarounds by OS components and count code lines of workarounds in each category.

Our study analyzes the distributions of OS-kernel developers and compilers, but does not infer any judgments for either role as a whole. For instance, RQ1 analyzes the knowledge flow directions. Even if a compiler bug is reported by an OS kernel developer, other OS kernel developers may not know about this compiler bug. It could be useful to recommend such compiler bugs to more developers, and we further discuss this issue in Section 7. As another example, RQ3 analyzes the time intervals from reporting compiler bugs to implementing their workarounds. Even if a kernel developer reports a compiler bug and implements its workaround in a short period, other OS-kernel developers may not know this compiler bug and may not implement its workaround so promptly. Instead of building a stereotype of OS-kernel developers, the purpose of our study is to search for possible research opportunities in reducing the negative impact of compiler bugs.

5 Empirical Results

5.1 RQ1: Knowledge Flow

5.1.1 Protocol. In this research question, we investigate knowledge flow directions between compilers and OS kernels. In one direction, OS kernel developers detect compiler bugs during their development and report them to the issue trackers of compilers. The knowledge flow is from

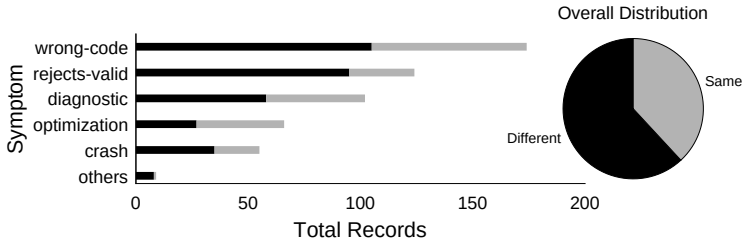


Fig. 3. Distribution of knowledge flow

OS kernels to compilers. In the other direction, OS kernel developers read compiler bug reports submitted by other before they implement workarounds. The knowledge flow is from compilers to OS kernels. The popularity of the two directions can reveal the typical workflow between compilers and OS kernels.

We implement a tool to extract the authors of workarounds and the reporters of compiler bugs. In particular, from the workarounds of the OS kernel, it extracts the names and email addresses of the committers, and from compiler bug reports, it extracts details, including reporters. We determine that the author of a workaround is the reporter of a compiler bug if their names are identical.

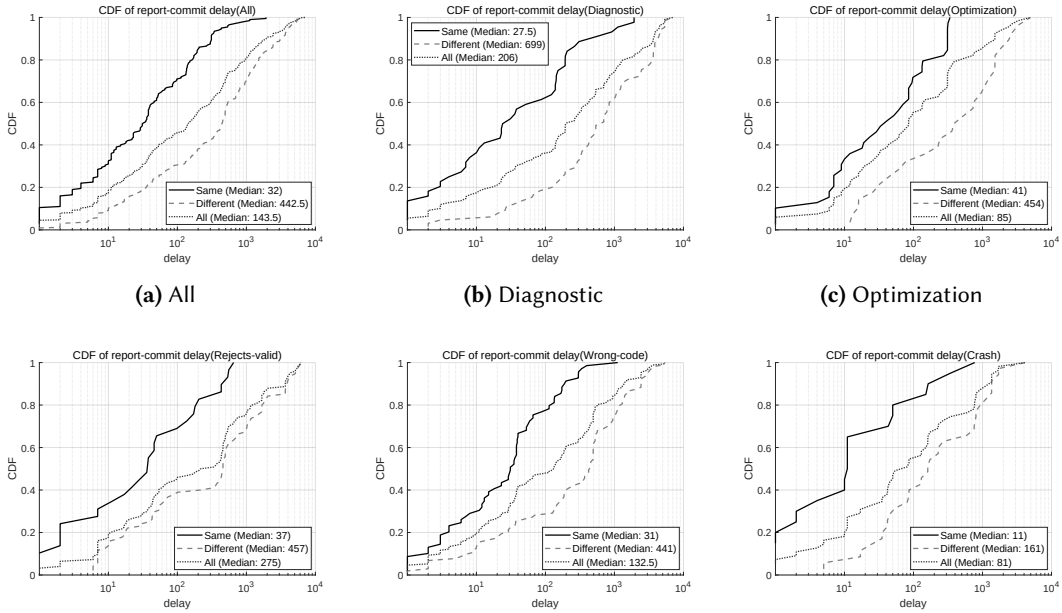
The symptoms of compiler bugs can affect the direction of the knowledge flow. When repairing bugs, compiler developers classify bugs by their symptoms. He and Zhong [31] merged the categories of `gcc` and `clang` and built a new taxonomy.In particular, wrong-code bugs occur when the generated machine code has a different semantics from the source code; rejects-valid bugs occur when compilers erroneously reject syntactically valid code; diagnostic bugs occur when compiler diagnostic messages are wrong; optimization bugs occur when compiler optimizations introduce program misbehavior; and crashes occur when compilers fail with internal crashes. All other compiler bugs are merged into the "other" category. Following their taxonomy, we grouped the knowledge flow directions by these symptoms.

5.1.2 Result. Figure 3 presents the results. The gray bars denote the cases where reporters of compiler bugs are the same as the programmers of OS kernel workarounds. For instance, as shown in Figure 2, the compiler bug [14] is reported by Yaf* Sha*, and the author of the workaround [17] is also Yaf* Sha*. Its commit message mentions the link to this compiler bug. In this case, the kernel developer may encounter an unknown compiler bug during development and report it to the compiler before handing it. The black bars denote the cases where the workaround authors and compiler-bug reporters are different. The results show that the knowledge from compilers to OS kernels is much more common than the reverse direction. The above observations lead to a finding:

Finding 1. 61.9% of the compiler bugs are not reported by programmers of OS workarounds.

This observation could be general to other types of software. Still, As the same programmer can use different accounts, we can overestimate the percentage, but such cases could be rare. As there are many compiler bugs, it is difficult for OS developers to identify which compiler bugs could affect their code. Therefore, it is useful to recommend compiler bugs to kernel developers.

Compared with their results, we find that wrong-code bugs appear more prominently in OS kernels. A plausible reason is that OS kernels contain more low-level, correctness-critical code, such as memory manipulation, bit operations, and architecture-dependent implementations, so



(a) All **(b)** Diagnostic **(c)** Optimization

(d) Rejects-valid **(e)** Wrong-code **(f)** Crash

Fig. 4. CDF from reporting to bypassing compiler bugs

incorrect compiler translations are more likely to cause visible failures and thus trigger workarounds. In addition, only for optimization bugs, the knowledge flow from OS kernels to compilers is more common than the reverse direction. This suggests that optimization bugs in OS kernels are more likely to be exposed by OS developers themselves. A plausible reason is that OS kernels heavily rely on aggressive optimizations, low-level coding idioms, and architecture-specific code, so optimization-related compiler bugs are more likely to be triggered first in OS development. The above observations lead to a finding:

In Figure 3, the vertical axis represents the symptoms of compiler bugs. The horizontal axis represents the number of links between OS workarounds and compiler bug reports. Among the symptoms, Figure 3 shows that wrong-code, rejects-valid, and diagnostic bugs are the top three symptoms, accounting for 32.8%, 23.4%, and 19.2% of the total bugs. As a comparison, He and Zhong [31] analyze compiler-bug workarounds in general projects. In their study, the wrong-code bugs account for 23.29% and rank only as the third top symptom. Compared with their results, we find that more wrong-code bugs affect OS kernels. OS developers pay more attention to wrong code bugs since they silently introduce unexpected behaviors. In addition, only in optimization bugs, the knowledge flow from OS kernels to compilers is more common than the reverse direction. This indicates that compared with other compiler bugs, OS developers identify more optimization bugs by themselves. Like wrong code bugs, optimization bugs also silently modify behaviors. The above observations lead to a finding:

Finding 2. OS kernel programmers implement more workarounds for wrong-code (account for 32.8%) and optimization (reported by OS programmers more) compiler bugs.

In summary, the knowledge flow from compilers to OS kernels is more common than the reverse direction. In the OS kernel, the top three symptoms of encountered compiler bugs are wrong-code, reject-valid, and diagnostic as other programmers, but OS programmers care more about wrong-code and optimization bugs.

5.2 RQ2: Time Interval

5.2.1 Protocol. In this research question, we investigate the temporal relationship between the commit dates of OS workarounds and the report dates of the mentioned compiler bug reports. Our tool extracts commit dates from GitHub and report dates from issue trackers. The reporting dates of compiler bug reports should be earlier than the commit dates of OS kernel workarounds. We then calculate the time interval from the reporting dates to the commit dates. We use the cumulative distribution function (CDF) to calculate the differences. The CDF formula is as follows:

$$F_T(t) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}_{\{t_i \leq t\}} \tag{1}$$

In this equation, $F_T(t)$ indicates probability of time interval within t days; n indicates the total number of records; $\mathbf{1}_{\{t_i \leq t\}}$ is indicator function equaling 1 if $t_i \leq t$, 0 otherwise, where t is the time interval we select to know its distribute location in all records and t_i is the i-th time interval data in all records. For instance, as shown in Figure 2, the compiler bug was reported on Dec/4/2024, and the workaround was submitted on Dec/5/2024. The time interval is 1 day. In 100 records, if there are 10 records with a time interval within 1 day, the CDF will be 0.1.

As we introduced in Section 5.1.1, the knowledge flows between compiler bug reports and OS workarounds have two directions, and the compiler bugs have different symptoms. When analyzing the time intervals, we take the directions and the symptoms into consideration and report their impact.

5.2.2 Result. Figure 4 shows the results. The horizontal axes denote the time intervals in days. The vertical axes denote the CDFs. The dotted lines show the results of all data; the solid lines show the results when bug reporters and workaround authors are the same; and the dashed lines show the results when bug reporters are not workaround authors.

In Figure 4a, when the delay is 100, the solid line shows that the corresponding CDF is 0.7, indicating that when the bug reporter and workaround author are the same, 70% of the data has less than 100 days of delay. For the dashed line, the corresponding ordinate is about 0.3, indicating that when the bug reporter is different from the workaround author, only 30% of the data has less than 100 days of delay. The median of the dotted line is 143.5 days, the median of the solid line is 32 days, and the median of the dashed line is 442.5 days. The observation leads to a finding:

Finding 3. When compiler bugs are not reported by OS programmers, it takes 10 times more days to identify and bypass the impact of them in OS.

When compiler bugs are identified by other programmers, their handling time is much longer. Indeed, before a compiler bug is identified, it could already affect OS kernels. As a result, we can underestimate the time intervals, and there is a strong need to identify whether compiler bugs affect the code under development.

Figure 4 presents the results categorized by symptoms. In compiler testing, most approaches detect crashes since such bugs are more visible. Finding 2 reports that OS programmers pay more

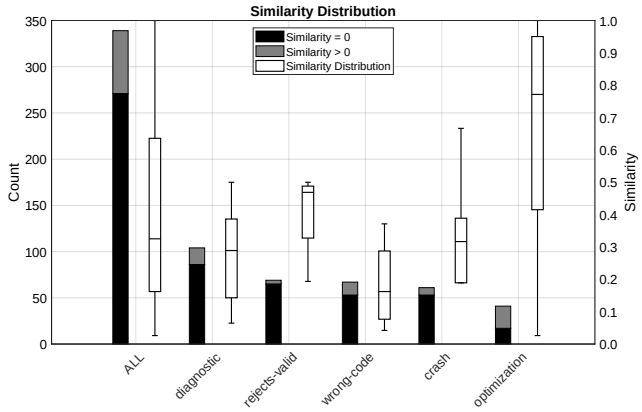


Fig. 5. The similarity between code sample and real code

attention to optimization and wrong-code bugs. As a result, crashes, optimization, and wrong-code bugs have shorter gaps, and their medians are 81, 85, and 132.5 days, repetitively. As for the time intervals between two knowledge directions, the two symptoms also have fewer gaps. The observations lead to a finding:

> **Finding 4.** The time intervals between reporting compiler bugs and bypassing them in OS kernels are shorter when the symptoms are crash (median 81 days), optimization (median 85 days), and wrong-code (median 132.5 days) compared with overall data (median 143.5 days).

In summary, the time interval between reporting compiler bugs and bypassing them in OSes is 10 times slower if compiler bugs are not reported by OS programmers. If compiler bugs are crashes, optimization, and wrong-code bugs, the time intervals are shorter than other symptoms.

5.3 RQ3: Code Similarity

5.3.1 Protocol. Our findings in RQ2 indicate that it is desirable if a tool can determine whether a compiler bug affect a software project. In this RQ, we explore the challenges of implementing such a tool. Intuitively, the buggy code in bug reports should be similar to the triggering buggy programs in real development, but the similarity will be reduced after programmers reduce triggering source files to code examples in bug reports. To explore to what degree this intuition holds, we calculate the similarity between the code sample in compiler bug reports and the source code in OS kernels.

We use the JPlag [36] to calculate the similarity between these two source files. As shown in Table 1, our collected OS workarounds mention 320 compiler bug reports. 161 of such bug reports provide 162 code samples in test cases or comments. These bugs are mentioned in 249 OS workarounds. Among them, 204 workarounds modify source files, but nine of such workarounds modify source files that only contain macro definitions, *e.g.*, `#define`. JPlag [36] is unsuitable to analyze such source files since it ignores macro definitions. In three workarounds, a modified file has more than 7,000 lines of code. JPlag [36] throws an exception when it is used to compare such large files. As a result, we remove the three workarounds. The remaining 192 workarounds modify 339 source files suitable for similarity calculation. For these files, we use the GitHub API to extract their pre-commit versions. We then compute the similarity between the code samples extracted from the bug reports and the pre-modification source codes. Consequently, this RQ quantifies

```
49  int kvm_vcpu_read_guest();
50  static void kvmhv_update_ptbl_cache(struct kvm_nested_guest *gp) {
51      struct kvm_nested_guest __trans_tmp_14 = *gp;
52          __srcu_read_unlock(__trans_tmp_14);
53  }
54  void byteswap_pt_regs(struct pt_regs *regs) {
55      unsigned long *addr = (long *)regs;
56      for (; addr < (unsigned long *)(regs + 1); addr++)
57          *addr = __builtin_bswap64(*addr);
58  }
59  int kvmhv_read_guest_state_and_regs(struct hv_guest_state *l2_hv,
60                                      struct pt_regs *l2_regs) {
61      return kvm_vcpu_read_guest(l2_hv) || kvm_vcpu_read_guest(l2_regs);
62  }
```

Bug Sample Code(llvm49610.cpp)

Similar code

```
...
57  /* Use noinline_for_stack due to https://bugs.llvm.org/show_bug.cgi?id=49610 */
58  static noinline_for_stack void byteswap_pt_regs(struct pt_regs *regs)
59  {
60          unsigned long *addr = (unsigned long *) regs;
62          for (; addr < ((unsigned long *) (regs + 1)); addr++)
63                  *addr = swab64(*addr);
64  }
...
```

OS Code(book3s_hv_nested.c)

(a) Same author

```
...
32  static inline size_t strlen(const char *p)
33  {
34      size_t ret;
35      size_t p_size = __builtin_object_size(p, 0);
36      if (p_size == (size_t)-1)
37          return __builtin_strlen(p);
38      ret = strnlen(p, p_size);
39      if (p_size <= ret)
40          fortify_panic(__func__);
41      return ret;
42  }
...
```

Bug Sample Code(gcc82365.cpp)

Similar Code

```
...
258 __FORTIFY_INLINE __kernel_size_t strlen(const char *p)
259 {
260     __kernel_size_t ret;
261     size_t p_size = __builtin_object_size(p, 0);
262
263     /* Work around gcc excess stack consumption issue */
264     if (p_size == (size_t)-1 ||
265         (__builtin_constant_p(p[p_size - 1]) && p[p_size - 1] == '\0'))
266             return __builtin_strlen(p);
267     ret = strnlen(p, p_size);
268     if (p_size <= ret)
269             fortify_panic(__func__);
270     return ret;
271 }
...
```

OS Code(string.h)

(b) Different authors

Fig. 6. Code samples whose similarity value is around 0.3

similarity across 339 distinct links between bug report code samples and the OS source code before modification.

For each link, we use the JPlag [36] to calculate the similarity between the code sample in the compiler bug report and the pre-modification code in the corresponding workaround. JPlag [36] uses Greedy String Tiling to iteratively find all maximal non-overlapping substrings. A tile (τ) denotes tokens shared by two token sequences. The code samples from bug reports are typically much smaller than the modified source files of OS workarounds. To make a fair comparison, we select the maximal similarity defined by JPlag [36]:

$$S(A,B) = \frac{\sum_{\substack{\tau \in \mathcal{T}(A,B) \\ |\tau| \geq k}} |\tau|}{\min(T_A, T_B)} \tag{2}$$

In the above equation, A and B denote the token strings of two code samples. T_A and T_B indicate the counts of the total tokens in A and B, respectively (i.e., $T_A = |A|$, $T_B = |B|$). $\mathcal{T}(A,B)$ is a set of maximally overlapped tiles. Each tile, τ, is a maximal substring shared by both A and B, and $|\tau|$ is its length. For instance, suppose that A has 150 tokens; B has 200 tokens; and their maximal substring has 50 tokens. $S(A,B)$ is $\frac{50+50}{150} = 66.7\%$. Only tiles whose lengths are more than k, *i.e.*, $|\tau| \geq k$, are considered in the calculation. Most code samples from compiler bug reports are small. The median of LOCs in such code samples is 13. In our study, we set k to be 6.

5.3.2 Result. Figure 5 shows the results. The horizontal axis denotes the symptoms of compiler bugs, the right axis denotes the similarity, and the left axis denotes the count of one workaround corresponding to one modified file. In the left bar, the black part represents the records with 0 maximal similarity, and the gray part represents the records with maximal similarity greater than 0. The right box plot represents the similarity distribution of records that have maximal similarity not equal to 0. As shown in Figure 5, the similarity values between the code sample of most bug reports and the modified code of OS kernels are zero. The code samples of bug reports are typically reduced and small, but the source files of OS kernels contain many irrelevant fragments. As a result,

even if they trigger the same compiler bug, a simple calculation of overlapped tokens is insufficient to identify their similarity in most cases. We draw box plots for links whose similarity values are more than zero. The medians are around 0.3. Figure 6 shows such examples. In particular, Figure 6b shows an example in which the bug reporter is different from the author of the corresponding workaround, and Figure 6a shows an example in which the bug reporter is the workaround author. In both cases, code samples from bug reports look similar to the modified files of OS kernels, but they have minor differences. For instance, in Figure 6a, the called methods have similar but different names, *i.e.*, `_builtin_bswap64()` and `swab64`. The above observations lead to a finding:

> **Finding 5.** Except for a few cases (68 in 339 cases), most (271 in 339 cases) of the code samples of bug reports are dissimilar to OS kernel code.

As discussed in Section 5.2, there is a strong need to determine whether a compiler bug affects a project. However, as most code samples are dissimilar, it is challenging to make such decisions.

As for symptoms, we find that the median of optimization bugs is the highest (around 0.8). After manual inspection, we find that 16 out of 24 related workarounds bypass the same gcc bug [1]. Due to this compiler bug, methods can be incorrectly uninlined. This report attaches a Linux kernel source file (`kernel/locking/mutex.c`) to illustrate the bug. To pass this bug, OS developers replace `inline` with `__always_inline` since the latter keyword forces compilers to inline methods. For instance, a modified code snippet is as follows:

```
- static inline int pageTail(struct page *page)
+ static __always_inline int PageTail(struct page *page)
```

As all the 16 workarounds have similar modifications, JPlag calculates high similarity values for these workarounds.

We also find that the median of the wrong-code bugs is lower than that of all the data. We manually analyze the results of all wrong-code compiler bug code samples to investigate the reason. For example, we study the gcc bug 101419 [4]. Compiler programmers mark this bug report as a wrong-code bug. The bug code sample in the report is:

```
union diffsize {
    int lg;
    char sm;
};
static void do_wipe(union diffsize *info){
    if (__builtin_object_size(&info->sm, 1) < sizeof(info->sm))
        __detected_overflow(__builtin_object_size(&info->sm, 1), sizeof(info->sm));
    __builtin_memset(&info->sm, 0, sizeof(info->sm));
    if (__builtin_object_size(&info->lg, 1) < sizeof(info->lg))
        __detected_overflow(__builtin_object_size(&info->lg, 1), sizeof(info->lg));
    __builtin_memset(&info->lg, 0, sizeof(info->lg));
}
```

GCC implements two built-in functions, such as `__builtin_memset()` and `__builtin_object_size()`, for the standard functions `memset()` and `sizeof()`, respectively. According to the bug report, this compiler bug is triggered when multiple `__builtin_memset()` methods are called to set the members (`int lg` and `char sm`) of the same union (`differsize`) with a given value. Due to this bug, `__builtin_object_size()` and `sizeof()` return different values. In a Linux workaround [6], the following code triggers this bug:

```
union ia_css_dvs_grid_u {
  struct ia_css_dvs_stat_grid_info dvs_stat_grid_info;
  struct ia_css_dvs_grid_info dvs_grid_info;
};
sh_css_pipe_get_grid_info(struct ia_css_pipe *pipe,
```

Table 2. OS kernel component

Component	Linux	FreeBSD	OpenBSD	Serenity	ReactOS	AROS	Zephyr
Driver	drivers/	sys/dev/	sys/dev/	Kernel/Devices/	base/	workbench/devs/	drivers/
Architecture	arch/	sys/{arch}	sys/arch/	Kernel/Arch/	hal/	arch/	arch/
Development	tools/	contrib/	gnu/	ToolChain/, AK/	sdk/tools/	tools/	boards/
Library	lib/, include/	lib/	lib/, include/	Kernel/Library	sdk/lib/	workbench/libs/	lib/, include/
File System	fs/	sys/fs/	sys/nfs/	Kernel/FileSystem/	drivers/filesystems/	workbench/fs/	subsys/fs/
Kernel Core	kernel/, mm/	sys/kern/	sys/kern/	Kernel/Memory/	ntoskrnl/	rom/kernel/	kernel/
Makefile	Makefile	Makefile	Makefile	CMakeLists	CMakeLists	Makefile	cmake/
Network	net/	sys/net/	sys/net/	Kernel/Net/	drivers/network/	workbench/network/	subsys/net/
Security	security/	sys/security/	sys/crypto/	Kernel/Security/	drivers/crypto/	rom/security/	subsys/secure_storage/

Note: Paths are relative to each OS source root directory. {arch} denotes architecture-specific subdirectories.

```
        struct ia_css_grid_info *info){
memset(&info->dvs_grid.dvs_grid_info, 0,
          sizeof(info->dvs_grid.dvs_grid_info));
memset(&info->dvs_grid.dvs_stat_grid_info, 0,
          sizeof(info->dvs_grid.dvs_stat_grid_info));
}
```

As the built-in functions are optimized, compilers often replace standard functions with their built-in functions during compilation. As a result, like the above code, calling standard functions can trigger compiler bugs involving built-in functions. The names of the union and its members can be different between code samples and real code. Unlike other types of compiler bugs, the bug reports of wrong code bugs typically include test programs to checking whether compilation results are correct. As real code is unlikely to provide such test programs, the similarity values become lower. Due to the above observations, the code samples of wrong code bugs are more different than other types of compiler bugs. Consequently, we derive a finding:

Finding 6. The code samples of wrong-code bugs (with the median similarity of 0.16) are more different from the modified files of OS kernels than compiler bugs with other symptoms (with the median similarity of 0.33).

In summary, due to the difference between the bug reproduction environment and the real OS development environment, most of the bug code samples will not show obvious similarity with the real OS code. In a small number of similar cases, the bug code samples are similar but still different compared with the modified files in OS kernels. In terms of symptom distribution, due to the complexity and characteristics of the wrong-code symptom itself, the similarity between the bug code samples under this symptom and the real OS code related to this bug will be lower than that of other symptoms and the overall data.

5.4 RQ4: Association

5.4.1 Protocol. In this research question, we investigate the associations among the symptoms of compiler bugs, the affected OS components, and the modified lines of code of OS workarounds. We analyze all the links in Table 1. We use the top-level folder names of modified source files to determine the affected components. We inspect the file structures of all OS kernels and merged their components. Table 2 shows our built taxonomy. The driver contains programs that enable communication between OS kernels and hardware devices. The architecture contains processor-specific implementations. The development includes tools for building and testing OS kernels. The library contains predefined functions that allow applications to access kernel features. The file system manages files and directories on storage devices. The kernel core contains the core implementations.

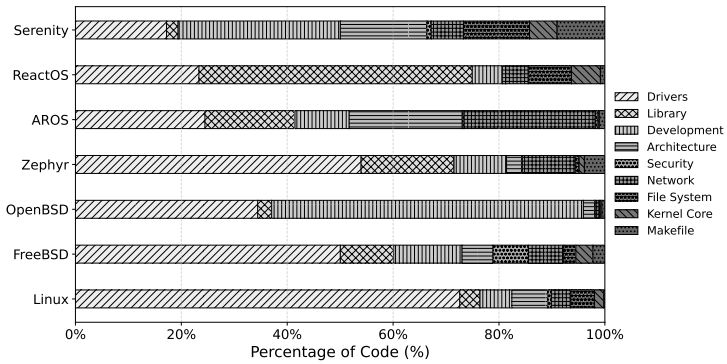


Fig. 7. Percentage of each component code

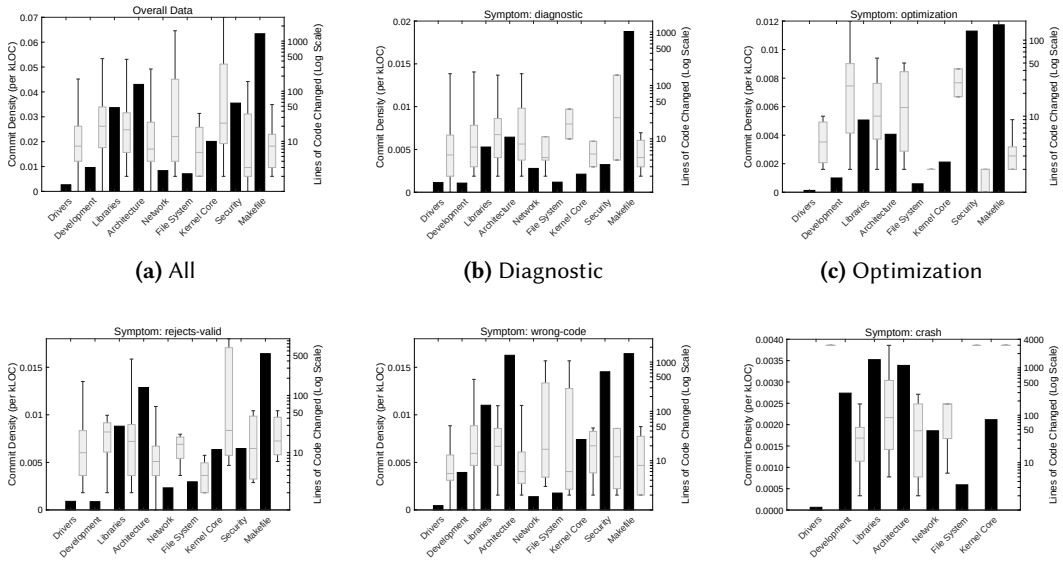
The Makefile defines build configurations. The network handles the communication with other computers. The security contains the implementations for protecting data. If a workaround modifies the file from a component, we consider that the component is affected. For instance, the gcc bug 117912 [8] is mentioned in a Linux workaround d938150 [10], and this workaround modified the file, `drivers/ata/ahci.h`. We determined that this compiler bug affects the driver component. If a workaround affects more than one component, we count it in each component. Figure 7 shows the percentage of each component code in each OS. The driver, the development, and the library account for more than others.

We count the workarounds on all the components in Table 2 and calculate the affected counts per thousand lines of code. For a workaround on each component, our tool uses the GitHub API to retrieve its added and deleted lines. The number of modified lines is the sum of deleted and added lines. To understand how compiler errors affect the development of an OS component, we draw box plots of modified lines for each component. To understand the impact of symptoms, we group the links by the symptoms of compiler bugs and draw box plots for each symptom.

We use modified code lines to measure the impact of compiler bugs, but this metric does not indicate the complexity of implementing workarounds since He and Zhong [31] report that some workarounds have repetitive changes.

5.4.2 Result. Figure 8 shows the results. The horizontal axes represent OS components. The left axes list the number of workarounds per thousand lines of code. The right axes list the modified lines of workarounds. For each component, the left black bar represents the number of workarounds that modify this component per thousand lines of code, and the right gray box plot shows the modified lines of the corresponding workarounds. Figure 8a shows the overall results. Although the drivers have the most code lines, it has the fewest workarounds for compiler bugs. The code from the driver can have simpler structures and is unlikely to trigger compiler bugs. As it defines build configurations, the Makefile component has 63 workarounds per million lines. The architecture component also has more workarounds (43 workarounds) per million lines than other symptoms. This is becauss the architecture component relies heavily on low-level, architecture-dependent code, which are sensitive to compiler bugs. The above observations lead to a finding:

Finding 7. The architecture and the Makefile components have 43 and 64 workarounds per million lines respectively, which shows that they are the most affected per code line.



(a) All **(b)** Diagnostic **(c)** Optimization

(d) Rejects-valid **(e)** Wrong-code **(f)** Crash

Fig. 8. Commit density and modified code lines of workaround grouped by OS kernel component

In Figure 8a, the box plots show the modifications of workarounds. The workarounds on the development and library components have more modifications. The modifications of all components typically involve around 10 lines of code. The result is consistent with Zhong [45]. Although compiler bugs are less likely to affect the kernel core, once it is affected, the workarounds typically involve more modifications. Zhong [45] reported that bypassing some compiler bugs needs repetitive and systematic changes. We also found that workarounds in OS kernels also have repetitive changes. For instance, a workaround [9] of Linux bypasses a wrong-code compiler bug [5]. From this workaround, we find many repetitive replacements such as:

```
include/uapi/linux/ethtool.h
@@ -322,7 +322,7
- __u8 data[0];
+ __u8 data;
include/uapi/linux/jffs2.h
@@ -155,7 +155,7
- __u8 data[0];
+ __u8 data;
```

In this example, the compiler bug [5] caused erroneous object size calculations for legacy zero-length arrays. This prompts a strict mode to enforce semantic distinction. This strict enforcement necessitated the standardization of all legacy zero-length arrays to flexible array members. Consequently, many repetitive changes are needed to bypass this compiler bug. Such compiler bugs can influence on the OS kernel and other software projects. Therefore, we have made similar findings to Zhong [45] in this regard.

To formally test our observation, we compare the per-commit total modified lines of each target component (*Development*, *Libraries*, and *Kernel Core*) against all other components. Because modification-line data are highly skewed and vary in sample size across components, we applied the nonparametric Mann–Whitney U test with Holm's correction for multiple comparisons, and report Cliff's δ as the effect size. Table 3 shows the summary.

Table 3. Nonparametric test results for target components

Component	Median	Cliff's δ	p_{Holm}
Development	22	0.20	<0.001
Library	16	0.15	0.057
Kernel Core	21.5	0.22	0.12

Note: Medians are lines changed per commit; Cliff's δ indicates effect size.

The *Development* component shows significantly larger modifications ($p_{\text{Holm}} < 0.001$), and *Libraries* exhibits a similar trend ($p_{\text{Holm}} \approx 0.057$). Although *Kernel Core* has fewer commits, its median and effect size remain high, implying large but infrequent workarounds. These results confirm that workarounds on the development and the library involve more modifications, while workarounds on the kernel core are rarer but tend to be extensive. The above observation leads to a finding:

Finding 8. The workarounds on the development, the library, and the kernel core have more modifications, and some modifications are repetitive. They medianly need 22, 16 and 21.5 lines of modification than 10 lines of other components.

Figure 8 shows the results grouped by symptoms. Although modifying the Makefile component bypasses all the other symptoms, we find no modifications to the Makefile when programmers bypass crashes. Our result indicates that modifying source files is the only observed way to bypass crash bugs. Besides the observation on Makefile, we find that the library and the architecture components are frequently modified to bypass all symptoms. These components often employ advanced language constructs, inline assembly, or architecture-dependent optimizations. This increases their susceptibility to various symptoms of bugs. Besides the two components, each symptom of compiler bugs affects specific components. In particular, optimization bugs often affect the security component; reject-valid bugs affect the library and the architecture components more frequently; wrong-code bugs frequently affect the security, the library, and the architecture components; and crash bugs affect the library and the architecture more frequently.

Regarding the distributions of modified lines, the differences are generally minor, but fixes for crash bugs and wrong-code bugs require more modifications, particularly for crash bugs. This might be because crash bugs often originate from internal code flaws, needing deeper source code refactoring rather than simpler build file adjustments, which results in larger-scale modifications. These observations lead to a finding:

Finding 9. Modifying the Makefile is unlikely to bypass crash bugs, and bypassing crashes and wrong-code compiler bugs requires more modifications. They medianly need 33 and 12 lines of modification respectively compared with 10 lines of other bugs.

In summary, compiler bugs tend to affect larger components, and the Library and Makefile are the most affected if we consider the density. The workarounds to the development, the library, and the kernel core involve more modifications. As for symptoms, more symptoms affect the library and the development components, and bypassing crashes and wrong code requires more modifications.

6 Threats to Validity

The internal threats to validity include our technique to identify whether a bug report is an author of a workaround. In rare cases, a programmer can use different avatars or nicknames. As it is difficult to fully determine whether a bug reporter and a workaround author is identical, we can underestimate the cases when they are the same. The internal threats to validity also include the taxonomies of compiler bug symptoms and OS components. We build the taxonomies by merging those of programmers, but programmers may not define orthogonal taxonomies. Despite this threat, our taxonomies reflect how programmers handle bugs in the wild. For OS components, we mainly adopt the taxonomies of programmers, but programmers may not strictly classify source files by components. The internal threats to validity also include our static analysis. As it is difficult to reproduce compiler bugs, we do not inspect the runtime behaviors of compiler bugs. The threat could be minor since OS developers should have observed their runtime behaviors before implementing workarounds. We release our dataset for other researchers to check. The external threat to validity includes our limited subjects. This threat is shared by all studies and could be reduced with more OS kernels and compilers.

7 Interpretation

This section interprets our findings:

Recommending compiler bugs to OS kernel programmers. Finding 1 shows that in most workarounds, compiler bugs are not reported by OS kernel programmers. Finding 5.2.2 shows that it takes 10 times more time if compiler bugs are not identified by OS kernel programmers. Therefore, if a tool identifies whether reported compiler bugs are likely to affect OS code under development, it will reduce the time gap. However, Finding 5 shows that the code samples of bug reports can be similar to the source code of OS kernels, but overlapped tokens are insufficient to calculate their similarity in most cases. Researchers need more advanced techniques if they plan to identify such impact and recommend compiler bugs to OS kernel programmers. For instance, a recommendation tool can extract compiler versions and failure configurations from a bug report and combine them with revision histories to decide the impact on OS components. Based on the impact, the tool can recommend potentially relevant compiler bugs to the corresponding OS developers. In addition, Finding 2 reports that OS programmers pay more attention to optimizations and wrong-code bugs. OS programmers can be more interested if tools can recommend the two types of compiler bugs. Meanwhile, it can more visibly reduce the time intervals between reporting compiler bugs and bypassing them if tools can recommend other compiler bugs.

Bypassing compiler bugs based on our findings. Finding 7 shows that larger OS components like the library and build files are more affected by compiler bugs. OS programmers can pay more attention to these components. Finding 8 indicates that the workarounds of the development, the library, and the kernel core have more modifications. Systematic change tools [33, 35] can reduce the effort of implementing workarounds if they contain repetitive changes. Finding 9 shows that compiler bugs with different symptoms can affect specific components. If OS programmers are interested in a compiler bug, they can check the corresponding components more carefully. Researchers have proposed approaches to generate workarounds for CVEs [32], hardware errors [41], and web failures [26]. Our study reveals the relationship between OS components, symptoms, and workarounds. Based on our findings, researchers can propose approaches that generate workarounds for compiler bugs. Talebi *et al.* [40] proposed an approach to undo workarounds for kernel bugs. He and Zhong [31] contacted kernel programmers to remove workarounds for compiler bugs after such bugs are fixed, but kernel programmers refused to remove such workarounds since other

programmers can use the buggy versions of compilers in their development. It should be more cautious to remove such workarounds.

Repairing compiler bugs based on our findings. It takes much effort to repair compiler bugs. Zhong [45] reports that about half of compiler bugs are unfixed even if they are encountered in real development. Our findings are useful for compiler programmers to repair bugs more effectively. For instance, Finding 2 shows that wrong-code and optimization compiler bugs are more likely to attract the attention of OS programmers. Finding 4 shows that crashes, optimizations, and wrong-code compiler bugs have shorter time intervals from the reporting time of bug reports to the committing time of workarounds. Compiler programmers can prioritize bugs with these symptoms. Findings 5 and 6 show that the code samples of most bug reports are different from the OS code. If programmers check the differences, they can deepen their knowledge of compiler bugs and check whether existing patches truly fix compiler bugs. For compiler researchers, our findings can motivate them to build a detailed compiler bug taxonomy based on bug reports and workaround information, laying the foundation for automated diagnosis and intelligent recommended fixes.

Future work based on our findings. Finding 7, Finding 8, and Finding 9 show where OS workarounds happen and how large these changes are. Based on these findings, future work can build a fine-grained taxonomy of workaround strategies in OS kernels. The taxonomy can separate local edits, cross-file edits, and build-level edits. Finding 1 and Finding 5.2.2 show that many affecting compiler bugs are reported by non-OS programmers and are handled later in OS projects. Based on these findings, future work can reproduce these bugs in real OS code, real build settings, and specific compiler versions. This study can tell whether a bug has confirmed impact on the OS kernel. Finding 5 and Finding 6 show low source-level similarity between bug-report code and OS code. Based on these findings, future work can study semantic similarity at the IR level. For example, researchers can compare LLVM IR patterns, data-flow relations, and optimization passes instead of source tokens. This IR-level analysis can better explain why a compiler bug can affect OS code even when source syntax looks different.

8 Related Work

Empirical studies on Compiler bugs. Compiler bugs have been studied on a variety of compilers, such as deep learning compilers [30, 38], WebAssembly compilers [37], and common C compilers [39, 46]. The above studies analyze the characteristics of compiler bugs themselves, but our study analyzes how compiler bugs affect other software projects. Zhong [45] proposed a new angle to analyze compiler bugs by collecting compiler bugs in real development. He and Zhong [31] studied the real-world impact of compiler bugs. Following their ideas, we analyze the impact of compiler bugs, but we are the first to analyze how compiler bugs affect OS kernels.

Empirical studies on OS kernel bugs. Researchers have conducted various empirical studies on bugs in the OS kernel. These studies cover various perspectives such as semantics [27], conditional configuration options [23], and dependencies [24]. These studies analyze the characteristics of OS kernel bugs, but we analyze how OS kernels bypass compiler bugs. Xu *et al.* [43] report that compiler bugs can introduce bugs to OS kernels if kernel programmers fail to identify them. In contrast, we analyze how kernel programmers bypass compiler bugs if kernel programmers notice such compiler bugs.

Empirical studies on workarounds. Programmers implement workarounds to bypass software bugs. Lamothe *et al.* [34] analyze workarounds of API-related bugs. Ding *et al.* [29] analyze the workarounds for Python libraries. Their studies focus on API workarounds, but we focus on workarounds for compiler bugs. Yan *et al.* [44] analyze general workarounds. Most of their analyzed workarounds are irrelevant to compiler bugs. Our study complements the above studies with a detailed analysis of how OS kernels bypass compiler bugs.

9 Conclusion

Although there have been various studies on compiler bugs themselves, only two recent studies [31, 45] have started to analyze the impact of compiler bugs, and none have examined how such bugs affect OS kernels. To address this gap in a timely manner, we investigate the workarounds from OS kernels and examine how they bypass compiler bugs. Our study provides answers to four related research questions. Our study derives new findings. For instance, for the first time, we find that in most cases, the code samples in bug reports are quite dissimilar to the modified source files in OS kernels. Our findings are useful to motivate future research. For instance, as code samples are dissimilar, it is challenging for OS programmers to determine whether their code will be impacted by a given compiler bug. This observation motivates further research in this area and encourages compiler developers to better highlight the bugs they encounter. Our study also presents other insights that can deepen the understanding of compiler bugs and their influence on OS development.

Data Availability

More details are listed on our website: https://os-compiler-bug.netlify.app.

References

[1] 2015. GCC bug report 66122. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=66122.
[2] 2020. Bug 45802 - excessive stack usage for FORTIFY_SOURCE building the kernel. https://bugs.llvm.org/show_bug.cgi?id=45802.
[3] 2020. Linux commit b3b6a84. https://github.com/torvalds/linux/commit/b3b6a84c6a920c60fd1393c43818b3955441424b.
[4] 2021. GCC bug report 101419. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=101419.
[5] 2021. GCC bug report 101836. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=101836.
[6] 2021. Linux commit 69aa1de. https://github.com/torvalds/linux/commit/69aa1deeab47a47f1e7876fabb76e1e11496c418.
[7] 2021. serenity 73fab93 commit. https://github.com/SerenityOS/serenity/commit/73fab93ef582328f29a1b475e6da52f2fd68ab3d.
[8] 2022. gcc bug report 107917. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=107917.
[9] 2022. Linux commit e2b5421. https://github.com/torvalds/linux/commit/e2b542100719a93f8cdf6d90185410d38a57a4c1.
[10] 2022. Linux commit f077880. https://github.com/torvalds/linux/commit/f07788079f515ca4a681c5f595bdad19cfbd7b1d.
[11] 2022. llvm 14 warns with unaligned access for the packed struct. https://github.com/llvm/llvm-project/issues/55520.
[12] 2024. AROS. https://github.com/aros-development-team/AROS.
[13] 2024. freebsd. https://github.com/freebsd/freebsd-src.
[14] 2024. gcc bug report 117912. https://gcc.gnu.org/bugzilla/show_bug.cgi?id=117912.
[15] 2024. linux 2947a45 commit. https://github.com/torvalds/linux/commit/2947a4567f3a79127d2d540384e7f042106c1a24.
[16] 2024. Linux commit 81741e8. https://github.com/torvalds/linux/commit/81741e804ef4692e8a6741380352625397a3288b.
[17] 2024. Linux commit d938150. https://github.com/torvalds/linux/commit/d9381508ea2b590aff46d28d432d20bfef1ba64c.
[18] 2024. openbsd. https://github.com/openbsd/src.
[19] 2024. ReactOS. https://github.com/reactos/reactos.
[20] 2024. SerenityOS. https://github.com/SerenityOS/serenity.
[21] 2024. zephyr. https://github.com/zephyrproject-rtos/zephyr.
[22] 2 2024. Linux. https://github.com/torvalds/linux.
[23] Iago Abal, Claus Brabrand, and Andrzej Wasowski. 2014. 42 variability bugs in the linux kernel: a qualitative analysis. In *Proc. ASE*. 421–432.
[24] Mohamed F Ahmed and Swapna S Gokhale. 2009. Linux bugs: Life cycle, resolution and architectural analysis. *Information and Software Technology* 51, 11 (2009), 1618–1627.
[25] Randal E Bryant and David Richard O'Hallaron. 2011. *Computer systems: a programmer's perspective*. Prentice Hall.
[26] Antonio Carzaniga, Alessandra Gorla, Nicolò Perino, and Mauro Pezzè. 2015. Automatic Workarounds: Exploiting the Intrinsic Redundancy of Web Applications. *Transaction on Software Engineering and Methodology* 24, 3 (2015), 1–42.
[27] Haogang Chen, Yandong Mao, Xi Wang, Dong Zhou, Nickolai Zeldovich, and M Frans Kaashoek. 2011. Linux kernel vulnerabilities: State-of-the-art defenses and open problems. In *Proc. APSys*. 1–5.
[28] Huazhao Chen and Hao Zhong. 2026. How do Compiler Bugs Affect OS Kernels. In *Proc. ICSE*. ACM, Rio de Janeiro, Brazil, to appear.

[29] Hui Ding, Wanwangying Ma, Lin Chen, Yuming Zhou, and Baowen Xu. 2017. An Empirical Study on Downstream Workarounds for Cross-Project Bugs. In *Proc. APSEC*. 318–327.
[30] Xiaoting Du, Zheng Zheng, Lei Ma, and Jianjun Zhao. 2021. An empirical study on common bugs in deep learning compilers. In *Proc. ISSRE*. IEEE, 184–195.
[31] Zhixing He and Hao Zhong. 2025. From Bug Report to Workrounds:The Real-World Impact of Compiler Bugs. In *Proc. SANER*. to appear.
[32] Zhen Huang, Mariana DAngelo, Dhaval Miyani, and David Lie. 2016. Talos: Neutralizing Vulnerabilities with Security Workarounds for Rapid Response. In *Proc. S&P*. 618–635.
[33] Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. 2021. A systematic review of API evolution literature. *Comput. Surveys* 54, 8 (2021), 1–36.
[34] Maxime Lamothe and Weiyi Shang. 2020. When APIs are intentionally bypassed: an exploratory study of API workarounds. In *Proc. ICSE*. 912–924.
[35] Na Meng, Miryung Kim, and Kathryn S. McKinley. 2013. LASE: Locating and Applying Systematic Edits by Learning from Examples. In *Proc. ICSE*. 502–511.
[36] Philippsen M. Prechelt, Malpohl G. 2002. JPlag: Finding plagiarisms among a set of programs. *Journal of Universal Computer Science* 8, 11 (2002), 1016–1038.
[37] Alan Romano, Xinyue Liu, Yonghwi Kwon, and Weihang Wang. 2021. An empirical study of bugs in webassembly compilers. In *Porc. ASE*. 42–54.
[38] Qingchao Shen, Haoyang Ma, Junjie Chen, Yongqiang Tian, Shing-Chi Cheung, and Xiang Chen. 2021. A comprehensive study of deep learning compiler bugs. In *Proc. FSE*. 968–980.
[39] Chengnian Sun, Vu Le, Qirun Zhang, and Zhendong Su. 2016. Toward understanding compiler bugs in GCC and LLVM. In *Proc. ISSTA*. 294–305.
[40] Seyed Mohammadjavad Seyed Talebi, Zhihao Yao, Ardalan Amiri Sani, Zhiyun Qian, and Daniel Austin. 2021. Undo workarounds for kernel bugs. In *Proc. USENIX Security*. 2381–2398.
[41] Tsung-Po Liu, Shuo-Ren Lin, and Jie-Hong R. Jiang. 2013. Software Workarounds for Hardware Errors: Instruction Patch Synthesis. *Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32, 12 (2013), 1992–2003.
[42] Ziyuan Wang, Dexin Bu, Aiyue Sun, Shanyi Gou, Yong Wang, and Lin Chen. 2022. An empirical study on bugs in python interpreters. *Transactions on Reliability* 71, 2 (2022), 716–734.
[43] Jianhao Xu, Kangjie Lu, Zhengjie Du, Zhu Ding, Linke Li, Qiushi Wu, Mathias Payer, and Bing Mao. 2023. Silent bugs matter: A study of {Compiler-Introduced} security bugs. In *Proc. USENIX Security*. 3655–3672.
[44] Aoyang Yan, Hao Zhong, Daohan Song, and Li Jia. 2023. How do programmers fix bugs as workarounds? An empirical study on Apache projects. *Empirical Software Engineering* 28, 4 (2023), 96.
[45] Hao Zhong. 2025. Understanding compiler bugs in real development. In *Proc. ICSE*. 2368 – 2379.
[46] Zhide Zhou, Zhilei Ren, Guojun Gao, and He Jiang. 2021. An empirical study of optimization bugs in GCC and LLVM. *Journal of Systems and Software* 174 (2021), 110884.